

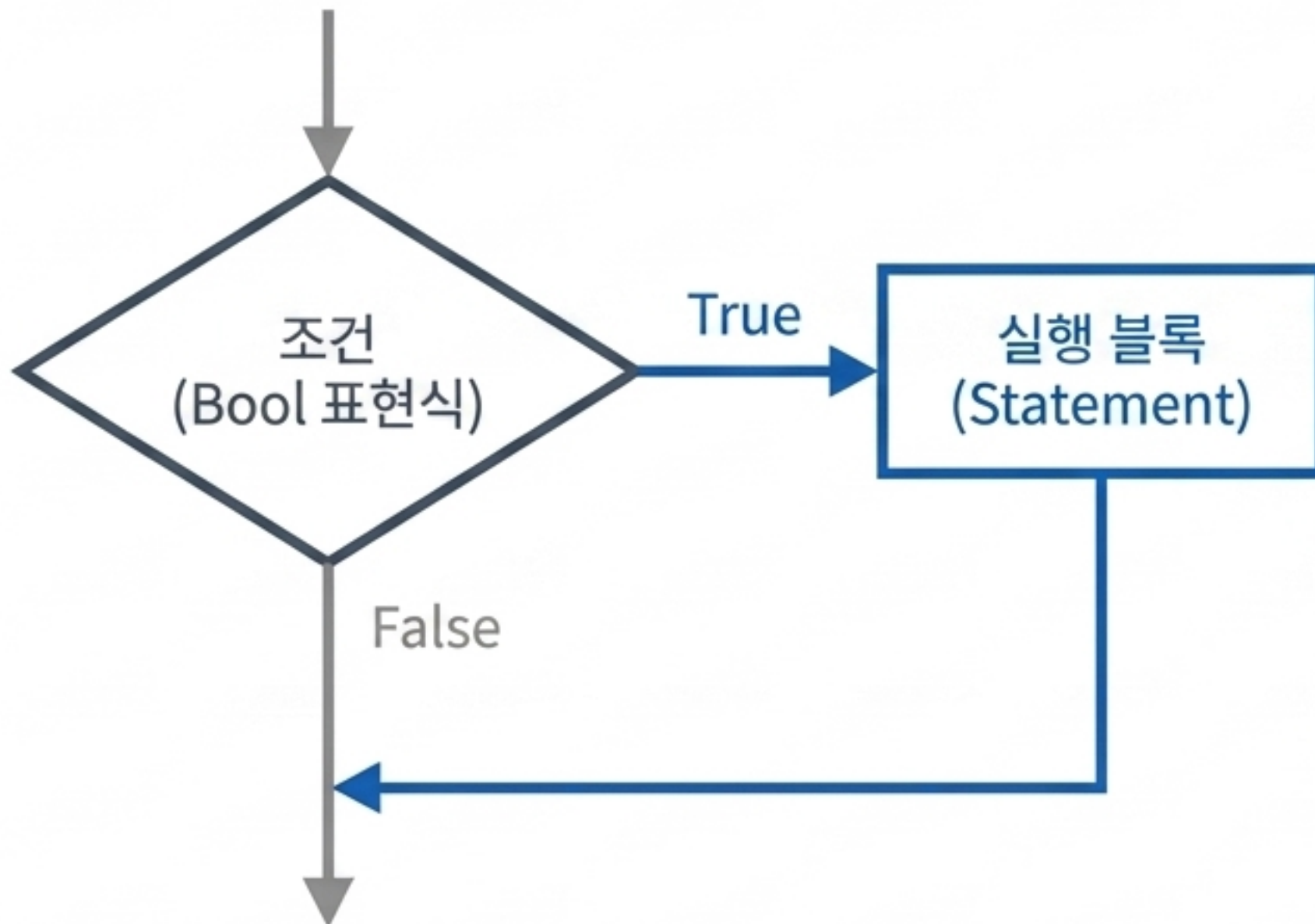


C# 제어의 기술: 조건문 완벽 가이드

기본 if문부터 고급 switch 로직까지, 코드의 흐름을 지배하는 방법

조건문이란?

불(Bool) 표현식의 결과(True/False)에 따라 프로그램의 실행 흐름을 나누는 스위치입니다.



코딩 규칙 (중괄호)

권장: 여러 줄이든 한 줄이든 중괄호 { }로 감싸기.

```
if (true) {  
    /* 코드 실행 */  
}
```

C# 기본 규칙: 첫 번째 줄에 if, 다음 줄에 중괄호 시작.

```
if (true)  
{  
    /* 코드 실행 */  
}
```

```
06  if (input % 2 == 0) ←
07  {
08      Console.WriteLine("짝수입니다!");
09  }
10  if (input % 2 == 1)
11  {
12      Console.WriteLine("홀수입니다!");
13  }
```

오직 조건이 참(True)일 때만
블록 내부가 실행됩니다.

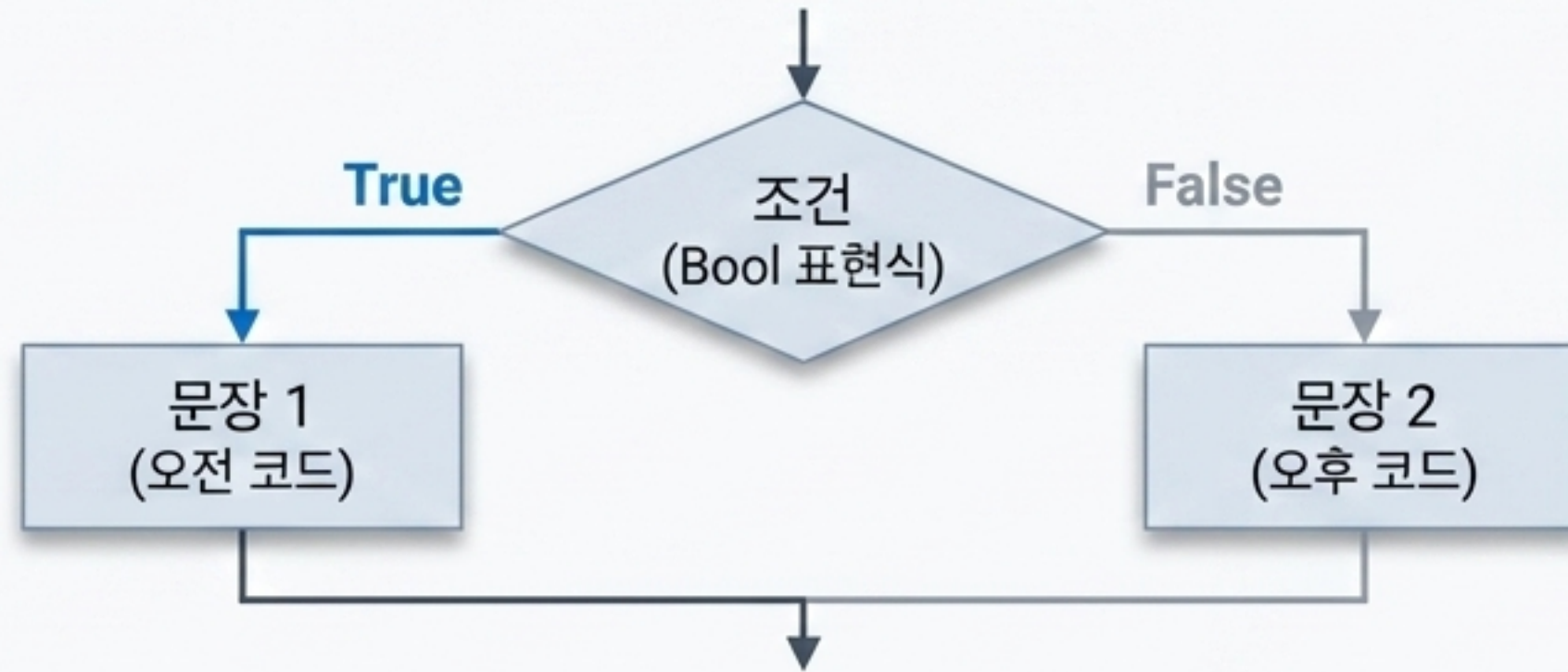
나머지 연산자(%)를
활용한 짝수 판별.

실행 결과

숫자 입력: 52273
홀수입니다!

두 갈래 길: if else 조건문

조건이 참이면 if 블록을, 거짓이면 else 블록을 실행합니다. 둘 중 하나는 반드시 실행됩니다.



```
if (DateTime.Now.Hour < 12)
{
    // 오전
}
else
{
    // 오후
}
```

현재 시간을 0~23의 정수로 반환하는 C# 내장 기능.

❌ 중첩 조건문: 가독성 저하

```
if (DateTime.Now.Hour < 11)
{
    Console.WriteLine("아침 먹을 시간입니다.");
}
else
{
    if (DateTime.Now.Hour < 15)
    {
        Console.WriteLine("점심 먹을 시간입니다.");
    }
    else
    {
        Console.WriteLine("저녁 먹을 시간입니다.");
    }
}
```

✅ if else if 조건문: 깔끔한 분기 처리

```
if (DateTime.Now.Hour < 11)
{
    Console.WriteLine("아침 먹을 시간입니다.");
}
else if (DateTime.Now.Hour < 15)
{
    Console.WriteLine("점심 먹을 시간입니다.");
}
else
{
    Console.WriteLine("저녁 먹을 시간입니다.");
}
```

중복되지 않는 3가지 이상의 조건을 구분할 때는 else if를 사용하여 코드의 깊이(Depth)를 줄이세요.

오류: 수학적 범위 표현

CS0019: '<' 연산자는 'bool' 및 'double' 형식의 피연산자에 적용할 수 없습니다.

```
if (4.0 < score < 4.5)
```

수학식 $4.0 < \text{score} < 4.5$ 는 C#에서 컴파일 오류를 발생시킵니다.

해결: 논리 연산자 사용

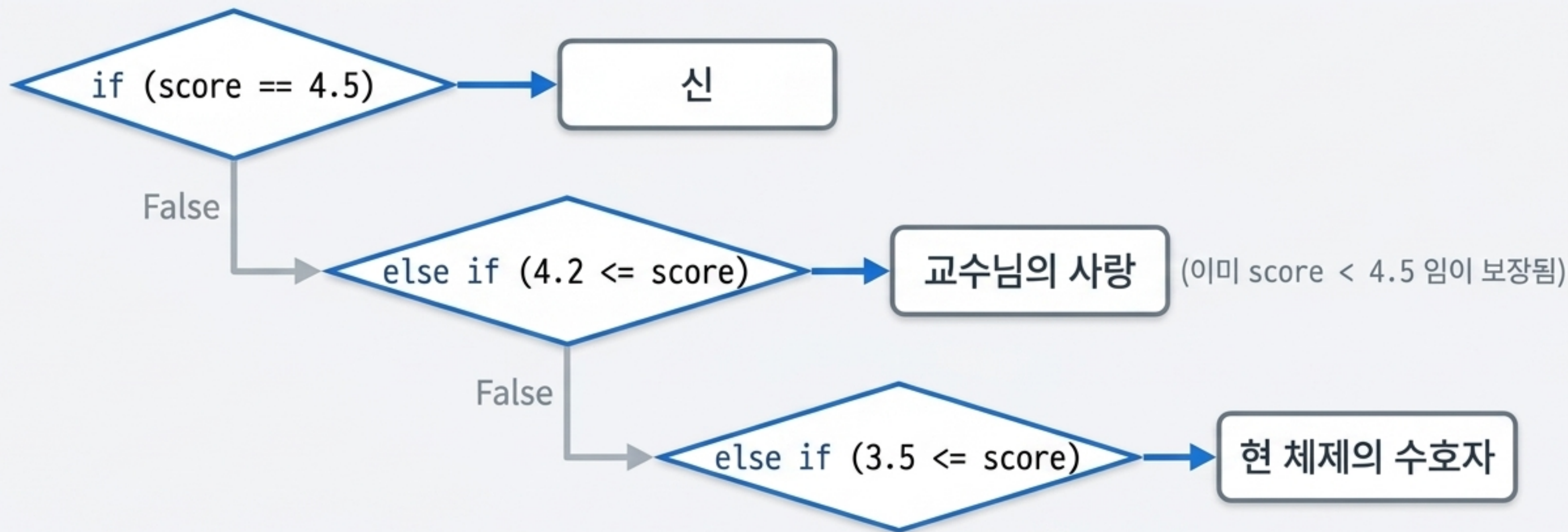


```
if (4.0 < score && score < 4.5)
```

논리 연산자 &&(AND)를 사용하여 두 조건을 결합해야 완벽한 범위를 지정할 수 있습니다.

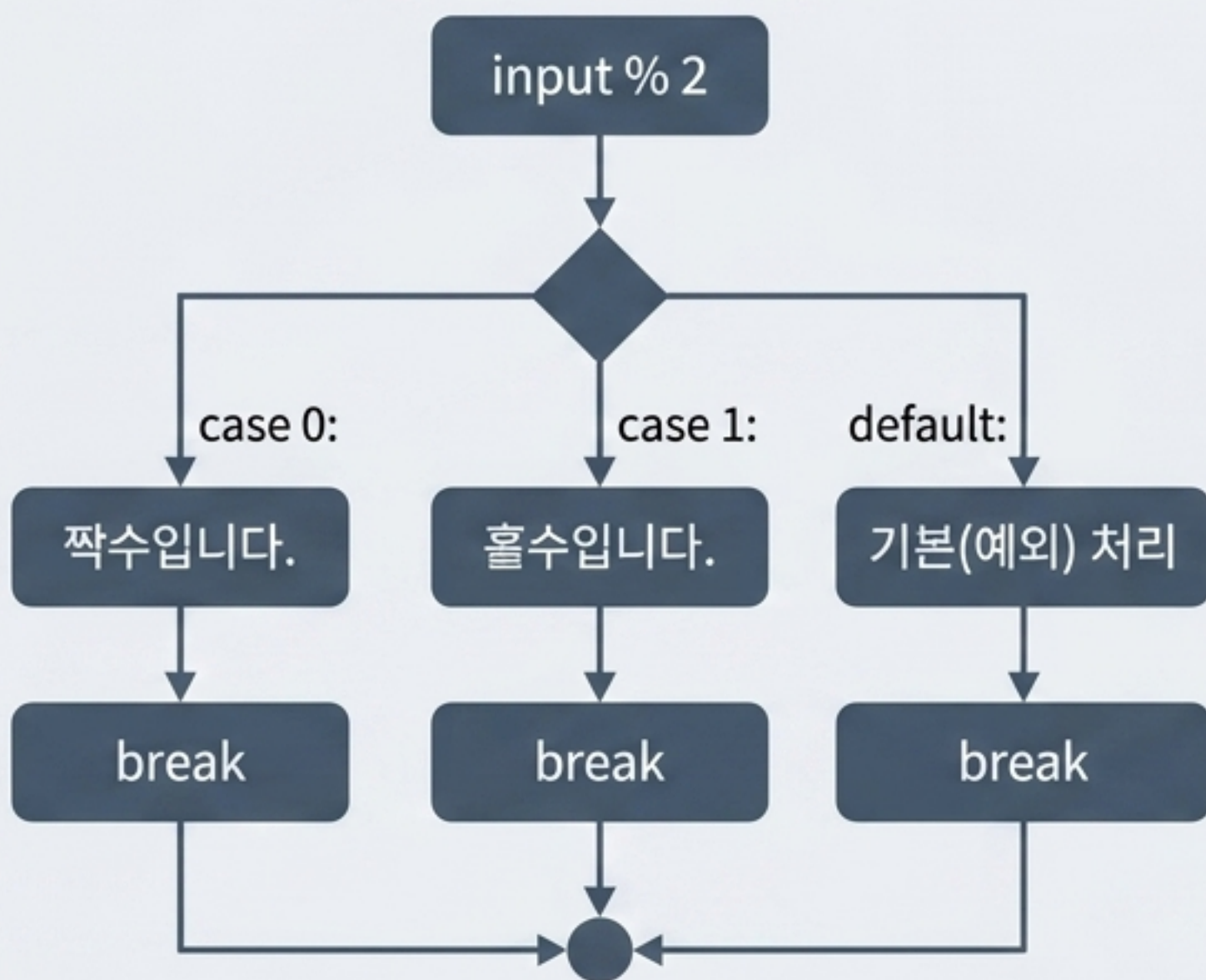
폭포수 효과: 조건문의 최적화

이전 조건이 거짓(False)임이 증명되면, 다음 else if에서는 그 범위를 다시 검사할 필요가 없습니다. 코드가 훨씬 간결해집니다.



정밀 타격: switch 조건문

범위가 아닌 특정 값을 정확히 비교할 때 사용합니다.

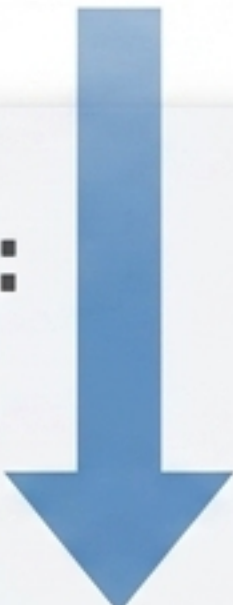


```
switch (input % 2) ← 기준 값
{
    case 0: ← 일치하는 케이스
        Console.WriteLine("짝수입니다.");
        break;
    case 1:
        Console.WriteLine("홀수입니다.");
        break;
    default: ← 기본(예외) 처리
        Console.WriteLine("예외 케이스입니다.");
        break;
}
```

break의 중력과 의도적 생략

break 키워드가 없으면 실행 흐름은 다음 case로 계속 떨어집니다(Fall-through).

이를 활용해 여러 값이 동일한 결과를 낼 때 코드를 압축할 수 있습니다 (예: 12월, 1월, 2월은 모두 겨울).



```
case 12:
case 1:
case 2:
    Console.WriteLine("겨울입니다.");
    break;
```

불_표현식 ? 참일_때_값 : 거짓일_때_값

1. 조건을 묻는다
(Question)

2. 참이면
이 값을 반환

3. 거짓이면
이 값을 반환

삼항 연산자(Ternary Operator)란?

피연산자가 3개인 유일한 연산자입니다.
짧은 조건에 따른 변수 할당을 단 한 줄로 끝냅니다.

```
Console.WriteLine(number > 0 ?  
    "자연수" : "아닙니다");
```

응용 1: 메서드의 반환값 활용

문자열 검색 메서드인 Contains()는 결과로 true 또는 false를 반환합니다.
따라서 if문의 괄호 () 안에 완벽하게 들어맞습니다.

```
if (line.Contains("안녕")) {  
    Console.WriteLine  
}
```

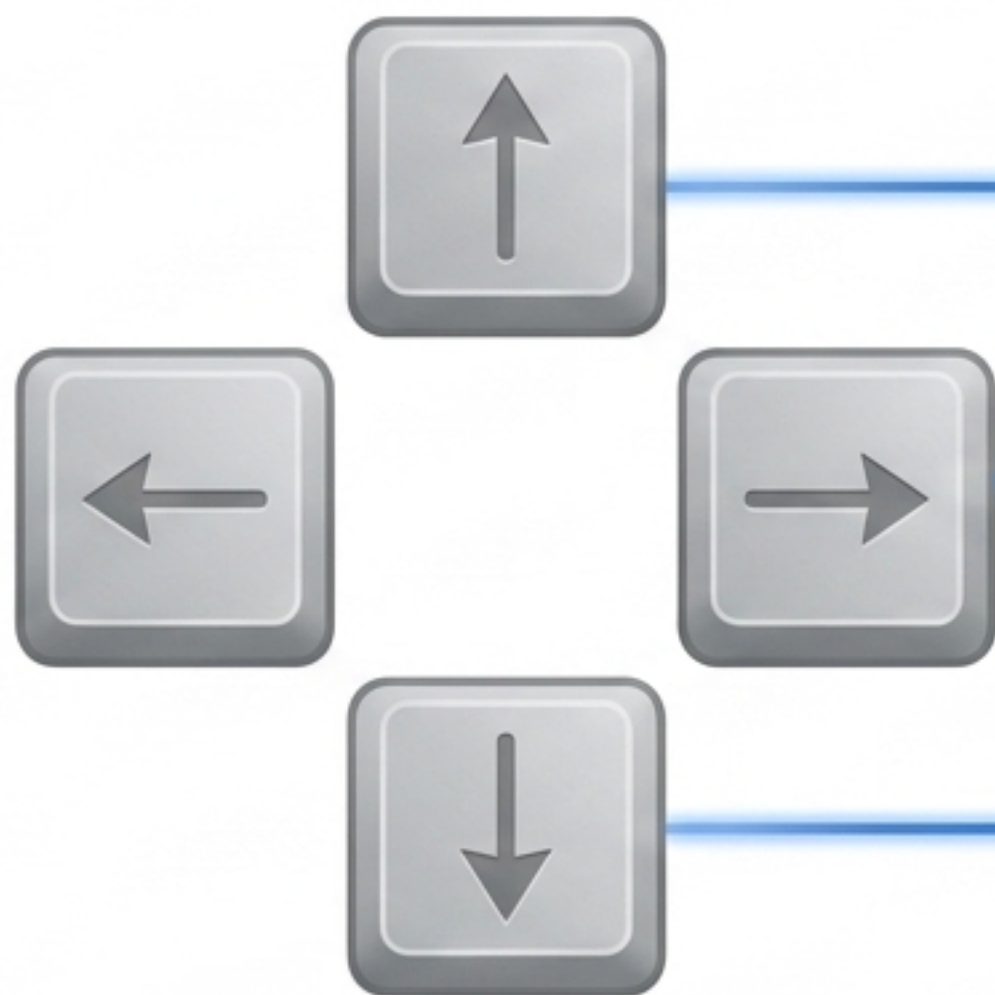
bool string.Contains(**string** value)

이 문자열 내에서 지정한 하위 문자열이 발생하는지를 나타내는 값을 반환합니다.

value: 검색할 문자열입니다.

응용 2: 키보드 입력 제어 (UI/게임 로직)

Console.ReadKey()로 사용자의 입력을 받고, ConsoleKey 열거형과 switch문을 결합하면 직관적인 컨트롤러를 만들 수 있습니다.



```
ConsoleKeyInfo info = Console.ReadKey();
switch(info.Key)
{
    case ConsoleKey.UpArrow:
        // Logic for UpArrow
        break;
    case ConsoleKey.RightArrow:
        // Logic for RightArrow
        break;
    case ConsoleKey.DownArrow:
        // Logic for DownArrow
        break;
    case ConsoleKey.LeftArrow:
        // Logic for LeftArrow
        break;
}
```

어떤 조건문을 사용해야 할까요?

	if - else if 범위와 복잡한 논리	switch 특정 값의 정밀 일치	? : 조건 연산자 간단한 인라인 할당
사용 목적	다목적/범위 연산	다중 값 분기	변수 초기화
코드 형태	중괄호 블록	case와 break	단일 라인
최적의 사용처	학점 계산(범위)	키보드 입력(정확한 값)	단순 훌쩍 판별 후 변수 저장

1. { 구조의 명확성 }

한 줄의 코드라도 중괄호를 생략하지 마세요. (안전성)

2. 폭포수의 원리

else if는 이전 조건을 기억합니다. 중복 검사를 피하세요. (효율성)

3. 도구의 매칭

범위는 **if**, 값은 **switch**, 한 줄 할당은 **?:** 연산자를 활용하세요. (적합성)